

Noetia — Documentación de Project Management

Versión 1.0 | Mayo 2026

1. Acta de Constitución del Proyecto

Información del Proyecto

Campo	Valor
Nombre del Proyecto	Noetia — Plataforma de Lectura Multimodal
Fecha de Inicio	2 de junio de 2025
Fecha de Finalización	25 de mayo de 2026
Duración	12 meses
Estado	✅ Beta lanzada, envío al App Store en progreso

Objetivos del Proyecto

- Diseñar, construir y lanzar una plataforma de lectura multimodal lista para producción para audiencias hispanohablantes
- Entregar sincronización texto-audio frase por frase en web y móvil
- Construir un motor de monetización (suscripciones + tokens) capaz de procesar pagos reales vía Stripe
- Lanzar una capa social (Clubes, compartir fragmentos, generación de tarjetas de citas) que cree bucles de retención de usuarios
- Desplegar en infraestructura de producción con CI/CD, monitoreo y backups automatizados

Criterios de Éxito

Criterio	Objetivo	Estado
Despliegue en producción activo	noetia.app	✅ Logrado
Cobertura de pruebas unitarias	≥ 80% en todos los servicios	✅ Logrado
Procesamiento de pagos con Stripe	End-to-end, transacciones reales	✅ Logrado
Apps móviles compilables	iOS + Android vía EAS	✅ Logrado
Migraciones de base de datos aplicadas	55 migraciones, cero rollbacks	✅ Logrado
Monitoreo y alertas	Grafana + Prometheus en vivo	✅ Logrado
Pipeline CI/CD	Auto-deploy al hacer push a main	✅ Logrado

Restricciones

- Presupuesto:** Bootstrapped (sin financiamiento externo). Costo de infraestructura < \$50/mes.
- Equipo:** 4 personas (PM, Backend, Frontend, DevOps) — sin contratistas adicionales.
- Tiempo:** Plazo firme de 12 meses para un producto listo para beta.

- **Tecnología:** Debe soportar uso offline en móvil, funcionalidades de clubes en tiempo real y despliegues automatizados.
-

2. Roles y Responsabilidades del Equipo

Product Manager (PM)

Responsabilidades principales:

- Definir y mantener el roadmap del producto
- Escribir historias de usuario y criterios de aceptación para cada sprint
- Dirigir todas las ceremonias Scrum (planning, review, retrospectiva, daily standup)
- Gestionar el backlog del producto (ordenamiento por prioridad, facilitación de estimaciones)
- Comunicar el progreso a los stakeholders
- Definir y hacer seguimiento de los KPIs
- Tomar decisiones de producto sobre UX, alcance de funcionalidades y trade-offs técnicos

Decisiones clave tomadas:

- Eligió el modelo de tokens estilo Audible sobre suscripción pura (permite precios por título sin limitar el catálogo gratuito)
 - Priorizó la funcionalidad de Clubes en Q4 a pesar del riesgo de alcance — identificó correctamente que es el principal motor de retención
 - Mantuvo las estadísticas de lectura como bucle de engagement inspirado en Duolingo (racha + metas semanales)
 - Definió la jerarquía de 3 prioridades: Experiencia del lector > Experiencia del autor > Biblioteca gratuita
-

Desarrollador Backend

Responsabilidades principales:

- Arquitectura e implementación de la API NestJS (endpoints REST, middleware, guards)
- Diseño del esquema PostgreSQL y escritura de todas las migraciones TypeORM
- Implementación de autenticación (JWT, Google, Facebook, Apple OAuth)
- Integración de Stripe (planes, webhooks, checkouts, suscripciones, tokens, tarjetas regalo)
- Construcción del sistema de heartbeat de estadísticas de lectura y cálculo de rachas
- Implementación del backend de Clubes (7 tablas: clubs, members, books, messages, discussions, polls, sessions)
- Escritura de pruebas unitarias para todos los servicios ($\geq 80\%$ de cobertura)

Stack a cargo: NestJS, TypeORM, PostgreSQL, Redis, Stripe SDK, Nodemailer, Meilisearch

Desarrollador Frontend

Responsabilidades principales:

- Construcción de la aplicación web Next.js (lector, biblioteca, fragmentos, compartir, perfil, facturación)
- Construcción de la app móvil React Native (iOS + Android) con paridad total de funcionalidades
- Implementación de la UI de sincronización frase por frase (resaltado de frase activa, auto-scroll)
- Construcción del modal de compartir tarjeta de citas (4 plataformas × múltiples formatos)
- Implementación del sistema i18n (español/inglés) en web y móvil con sincronización vía API
- Integración de Expo EAS para compilaciones móviles y actualizaciones OTA

- Escritura de pruebas unitarias para todos los componentes y hooks del frontend

Stack a cargo: Next.js 14, React Native (Expo SDK 51), Tailwind CSS, React Navigation, Zustand, AsyncStorage

Ingeniero DevOps

Responsabilidades principales:

- Aprovisionamiento y hardening del servidor de producción (Contabo VPS, Ubuntu 24.04)
- Configuración de Traefik v2.11 como reverse proxy con SSL automático Let's Encrypt
- Escritura y mantenimiento de todas las configuraciones Docker Compose (dev, overlay de prod, servidor)
- Construcción del pipeline CI/CD con GitHub Actions (test → build → deploy → migrate)
- Configuración de monitoreo Grafana + Prometheus (latencia de API, tasa de errores, salud de contenedores)
- Configuración de backups automáticos de base de datos (PostgreSQL diario, MinIO semanal)
- Gestión de seguridad del servidor (UFW firewall, fail2ban, puerto SSH no estándar 222)
- Configuración de buckets MinIO (libros/audio privados, imágenes públicas) y políticas de bucket

Stack a cargo: Docker, Docker Compose, Traefik, GitHub Actions, Grafana, Prometheus, cron de PostgreSQL, MinIO

3. Metodología del Proyecto

Framework: Scrum con sprints de 2 semanas **Duración del sprint:** 14 días **Total de sprints:** 24 sprints en 12 meses **Cadencia de sprint:** Inicio el lunes, fin el viernes (con review/retro el último viernes)

Ceremonias

Ceremonia	Frecuencia	Duración	Participantes
Daily Standup	Cada día de semana	15 min	Los 4
Sprint Planning	Cada 2 semanas (lunes)	2 horas	Los 4
Sprint Review	Cada 2 semanas (viernes)	1 hora	Los 4
Sprint Retrospectiva	Cada 2 semanas (viernes)	45 min	Los 4
Refinamiento de Backlog	Semanal (miércoles)	1 hora	PM + dev relevante
Revisión de Arquitectura	Mensual	2 horas	Backend + DevOps + PM

Herramientas

Herramienta	Propósito
GitHub Projects	Tablero Kanban, seguimiento de issues, backlog del sprint
GitHub	Repositorio de código, revisiones de PR, CI/CD
Slack	Comunicación diaria, threads de standup, alertas de incidentes
Figma	Diseño UI, wireframes, especificaciones de componentes

Notion	Wiki del proyecto, notas de reuniones, logs de retrospectivas
Loom	Actualizaciones de video asincrónicas para comunicaciones PM → stakeholders

Definición de Done (DoD)

Una historia está **terminada** cuando:

- ☐ La funcionalidad está implementada y funciona según los criterios de aceptación
- ☐ Existen pruebas unitarias en la ruta espejada bajo `tests/unit/`
- ☐ Las pruebas cubren el camino feliz, casos borde y escenarios de error
- ☐ Todas las pruebas pasan (`npm test` / `pytest`)
- ☐ La cobertura para el servicio modificado es $\geq 80\%$
- ☐ El código ha sido revisado y aprobado por al menos otro miembro del equipo
- ☐ No se introducen nuevos errores de lint o TypeScript
- ☐ La funcionalidad está desplegada en producción (vía auto-deploy CI/CD al hacer merge a main)
- ☐ El PM ha verificado la funcionalidad contra los criterios de aceptación

4. Historial de Sprints y Hitos

Q1: Fundación (Sprints 1-6 | Jun-Ago 2025)

Sprint 1-2 (2 jun – 27 jun) *Objetivo: Scaffolding del proyecto y entorno de desarrollo*

- Estructura del monorepo, Docker Compose (todos los 7 servicios), `.env.example`
- `init.sql` de PostgreSQL, configuración de Redis, configuración de buckets MinIO
- Inicio del proyecto NestJS, configuración TypeORM, primera migración (`CreateUsersTable`)
- Configuración de Jest y pytest en todos los servicios (umbrales de cobertura del 80%)
- GitHub Actions: lint + test + build CI en PRs

Sprint 3-4 (30 jun – 25 jul) *Objetivo: Sistema de autenticación*

- Autenticación por email + contraseña (registro, login, tokens JWT de acceso + refresh)
- Confirmación de email al registrarse (token de 24h vía Resend SMTP)
- Recuperación de contraseña vía enlace de reseteo
- Google OAuth, Facebook OAuth, Apple Sign-In
- Guards de autenticación, logout, wrapper de ruta protegida (web)

Sprint 5-6 (28 jul – 22 ago) *Objetivo: Libros y fundación del lector*

- Tabla de libros, endpoints CRUD, subida a MinIO de texto y audio
- Endpoint de streaming de audio (range requests)
- Diseño y almacenamiento del sync map a nivel de frase
- Endpoint de progreso de lectura (guardar/restaurar por índice de frase)
- Página de biblioteca (grid de libros), página de detalle de libro
- Layout del lector: texto + controles de audio, renderizado de frases

Hito Q1: Prototipo funcional del lector 

Q2: Plataforma Central (Sprints 7-12 | Sep-Nov 2025)

Sprint 7-8 (25 ago – 19 sep) *Objetivo: Motor de lectura completo*

- Resaltado de frase: sincronización de tiempo de audio → frase activa
- Clic en frase → seek de audio
- Cambios de modo sin interrupciones (lectura ↔ escucha ↔ escucha activa)
- Modo de escucha activa (audio + texto + resaltado en vivo)
- Controles de tamaño de fuente, modo oscuro, navegación por capítulos
- Preferencias de lectura (persistencia en localStorage)

Sprint 9-10 (22 sep – 17 oct) Objetivo: Sistema de fragmentos y compartir

- CRUD de fragmentos, panel Fragment Sheet
- Handler de selección de texto (mouse + touch)
- Servicio de generación de imágenes de tarjetas de citas (Python Flask + Pillow)
- Worker BullMQ para renderizado asíncrono de imágenes
- Templates de LinkedIn, Instagram, Facebook, Pinterest

Sprint 11-12 (20 oct – 14 nov) Objetivo: Fundación de monetización

- Integración Stripe: creación de clientes, sesiones de Checkout
- Planes de suscripción (Individual, Dúo, Familia — mensual + anual)
- Webhooks de Stripe: activar/cancelar suscripciones, emitir tokens en invoice.paid
- Sistema de tokens: creditBalance, redención, vencimiento a 90 días
- Compras de libros y guard de acceso
- Página de precios, UI de gestión de facturación

Hito Q2: Monetización en vivo, primeros pagos de prueba procesados ✓

Q3: Móvil y Social (Sprints 13-18 | Dic 2025-Feb 2026)

Sprint 13-14 (17 nov – 12 dic) Objetivo: Shell de la app React Native

- Inicio del proyecto Expo SDK 51, configuración del proyecto EAS
- Pantallas de autenticación: login, registro, login social (iOS + Android)
- Navegación por tabs, pantalla de biblioteca, detalle de libro
- Pantalla del lector: texto + sincronización de frases en móvil
- google-services.json, configuración de Firebase para FCM

Sprint 15-16 (15 dic – 9 ene) Objetivo: Funcionalidades móviles

- Captura de fragmentos en móvil y Fragment Sheet
- Enforcement del paywall en móvil (verificación de suscripción en cada login)
- Descarga offline de libros (frases cacheadas en AsyncStorage)
- Auto-sincronización al reconectar (hook NetInfo offline → online)
- Notificaciones push (Expo Notifications, APNs + FCM)
- Reproductor de audio móvil (Expo AV, modo en segundo plano, control de velocidad)

Sprint 17-18 (12 ene – 6 feb) Objetivo: Portal de autor + compartir en redes sociales

- Registro de autor/editorial y dashboard
- Endpoint de subida de libro (texto + audio + metadatos + portada)
- Enforcement de nivel de hosting (Básico: 1, Starter: 3, Pro: 12 libros)
- Vinculación de cuenta OAuth social (LinkedIn, Facebook, Instagram, Pinterest)
- Publicación directa desde la app a plataformas sociales
- ShareModal: selector de plataforma, selector de formato, selector de color hex, direcciones de gradiente

Hito Q3: Builds de iOS y Android corriendo en dispositivos físicos ✓

Q4: Clubes, i18n, Stats y Lanzamiento (Sprints 19-24 | Mar-May 2026)

Sprint 19-20 (9 feb – 6 mar) *Objetivo: Noetia Clubes — backend*

- 7 migraciones: clubs, club_members, club_books, club_messages, club_discussions, club_polls+votes, club_sessions
- CRUD de Clubes (crear, unirse, salir, ajustes, banear)
- Discusiones ancladas a frases vinculadas al phraseIndex del sync map
- Votaciones de clubes para nominaciones de libros
- Escucha Juntos: sesiones de escucha en vivo programadas
- Sistema de enlace de invitación al club

Sprint 21-22 (9 mar – 3 abr) *Objetivo: UI de Clubes + i18n*

- Página de descubrimiento de clubes, página del club (4 tabs: Chat, Discusión, Votaciones, Sesiones)
- Tutorial de Clubes (bottom sheet), punto de entrada WelcomeSplash
- i18n completo español/inglés en web y móvil
 - Selección de idioma al primer lanzamiento
 - Los 8 templates de email en dos idiomas (EN/ES)
 - ReaderTopBar, BottomNav, todos los 5 tutoriales completamente i18n
 - LanguageProvider sincroniza el idioma hacia/desde la API al montar

Sprint 23-24 (6 abr – 25 may) *Objetivo: Stats, privacidad, QA, preparación del lanzamiento*

- Sistema de estadísticas de lectura (heartbeat, gráfico de barras de 7 días, racha, metas)
- Configuración de privacidad (4 toggles con actualizaciones optimistas)
- Tabs de la página de perfil (Perfil / Stats / Privacidad)
- Monitoreo Grafana corregido (acceso Tailscale, corrección de alertas de falsos positivos)
- Pipeline CD reforzado (grupos de concurrencia, limpieza dinámica de contenedores)
- Configuración de EAS build (perfiles de producción, autoIncrement, canales OTA)
- 55 migraciones aplicadas en producción, lanzamiento beta

Hito Q4: Beta lanzada en <https://noetia.app> ✓

5. Métricas Clave del Proyecto

Velocidad (story points por sprint, promedio)

Trimestre	Velocidad Promedio	Notas
Q1 (Sprints 1-6)	28 SP	Rampa inicial; infraestructura pesada
Q2 (Sprints 7-12)	38 SP	Cadencia completa; funcionalidades complejas
Q3 (Sprints 13-18)	42 SP	Velocidad pico; paralelismo móvil
Q4 (Sprints 19-24)	35 SP	Complejidad de Clubes; sobrecarga de QA

Conteo de Entregables

Artefacto	Cantidad
Migraciones de base de datos	55

Endpoints de API	~85
Archivos de pruebas unitarias	35+
Componentes React (web)	40+
Pantallas React Native	12
Claves de traducción i18n	~400
Servicios Docker	9

6. Registro de Riesgos

Riesgo	Estado	Resolución
Complejidad de sincronización de frases subestimada	Mitigado	Buffer adicional asignado en Sprint 7-8
Confiabilidad de webhooks Stripe	Mitigado	Idempotency keys en todos los handlers de webhook
Casos borde de sincronización offline en móvil	Mitigado	NetInfo + cola de reintentos implementada
Caché de Docker causando migraciones obsoletas	Ocurrió	Paso de build --no-cache añadido; documentado en runbook
Corrupción de terminal SSH en servidor (pegado multilínea)	Ocurrió	Protocolo nano/base64 establecido; añadido a CLAUDE.md
Grafana 404 tras reinicio del servidor	Ocurrió	Solución provisional de inyección de cookie; acceso Tailscale configurado
Diferencias Apple Sign-In en iOS/Android	Mitigado	expo-apple-authentication lo maneja de forma nativa

7. Highlights de Retrospectiva

Lo que salió bien

- **Estructura monorepo con Docker Compose** hizo que los cambios entre servicios fueran rápidos y consistentes
- **CI/CD desde el Sprint 1** significó que cada merge era desplegable — sin "semana de integración" al final
- **Umbral de cobertura del 80%** detectó múltiples regresiones que habrían llegado a producción
- **Cambios de base de datos primero vía migración** significó cero drift de esquema entre entornos
- **TypeScript en todas partes** (API + web + móvil) redujo drásticamente los bugs entre capas

Qué se puede mejorar

- El alcance de Clubes fue mayor de lo estimado — debería haberse dividido en dos fases con un backlog separado de "Clubos v2"
- El i18n debería haberse diseñado en la UI desde el Sprint 1, no retroajustado en Q4

- El desarrollo móvil comenzó en Q3; una consideración más temprana habría influido en el diseño de la API
- DevOps debería haber configurado el tuning de alertas de Grafana antes para evitar falsos positivos

Mejoras de proceso implementadas

- Se añadió reunión de revisión de arquitectura tras la aparición de 7 migraciones interdependientes en el backend de Clubes
- Se introdujo la opción de standup asíncrono vía Loom para reducir la fatiga de reuniones en Q4
- Se creó CLAUDE.md como guía viva del desarrollador — se convirtió en el documento principal de onboarding del equipo

Documento mantenido por el Product Manager. Última actualización: 25 de mayo de 2026.